

🌉 BRIJ UNIVERSAL COMMERCE BRIDGE

Complete Technical Architecture & Multi-Platform Integration Specifications

****Document Classification:**** Technical Architecture — Production Ready

****Lead Architects:**** ADA ULTRA SI + TARANTULA + ANT QUEEN

****Date:**** January 11, 2026 | **Version:** 1.0 | **Status:** ✅ Production Ready Design

🎯 SYSTEM OVERVIEW

What is BRIJ?

****BRIJ**** (Bridge) is a universal commerce orchestration layer that enables merchants to sell through ALL major AI platforms via a single integration. Instead of building separate implementations for Google UCP, Microsoft Copilot Checkout, and OpenAI ChatGPT Instant Checkout, merchants integrate once with BRIJ and automatically gain access to 2.5+ billion users across all three ecosystems.

Core Value Propositions

****For Merchants:****

- ****70% cost reduction:**** Single integration vs. 3+ separate implementations
- ****80% faster deployment:**** 2-3 weeks vs. 12-18 months
- ****Universal reach:**** 2.5B+ users across Google, Microsoft, OpenAI
- ****Unified management:**** One dashboard for all platforms
- ****Automatic optimization:**** BRIJ routes to best-performing platform per user

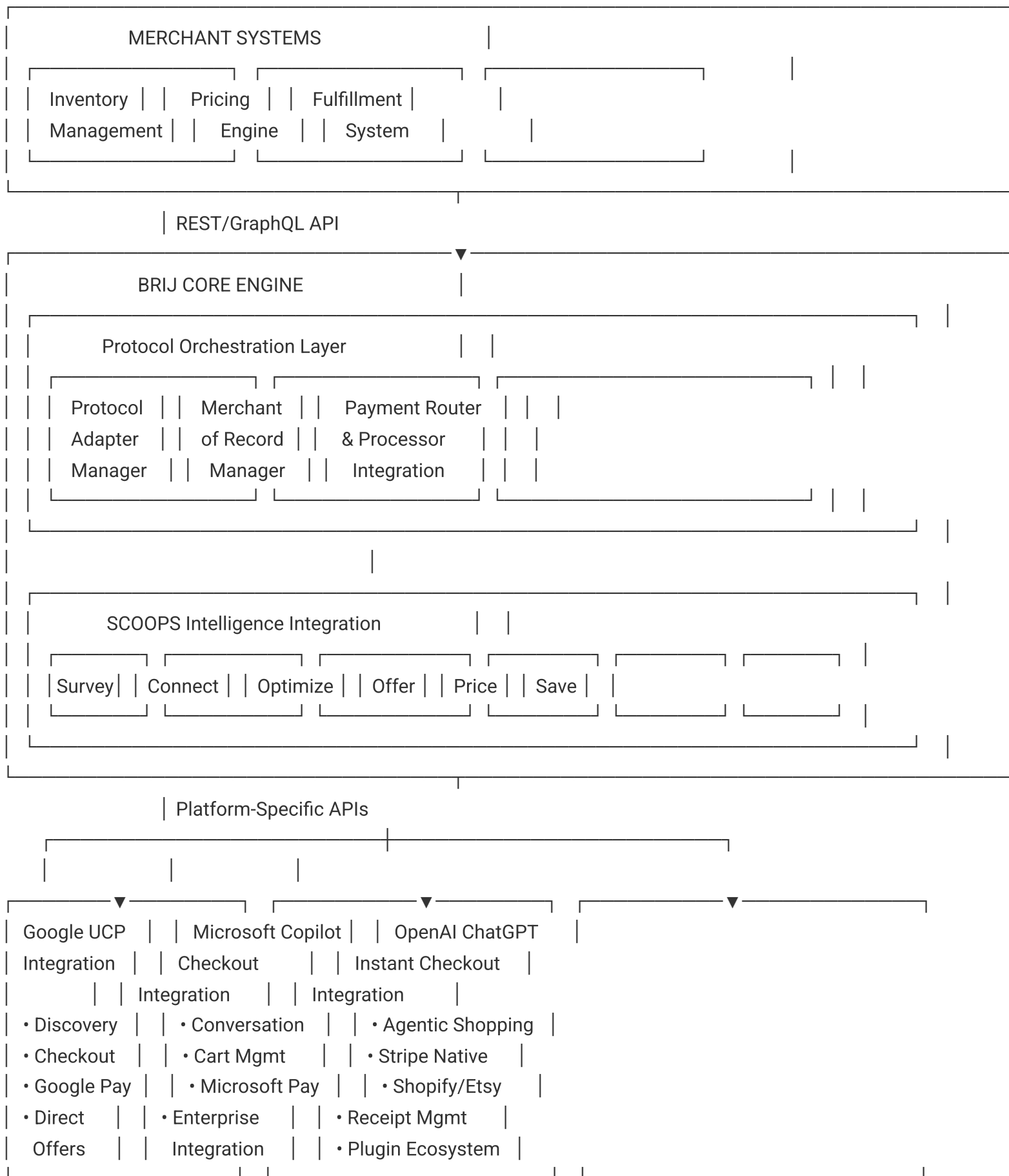
****For Consumers:****

- ****Seamless experience:**** Shop on any AI platform with consistent quality
- ****Best pricing:**** Automatic cross-platform price discovery
- ****Preferred payment:**** Use Google Pay, Microsoft Pay, PayPal, or any supported method
- ****Unified history:**** Purchase history tracked across all platforms

****For Platforms:****

- ****Merchant growth:**** Reduced integration friction = more merchants
- ****Standards compliance:**** BRIJ enforces protocol best practices
- ****Revenue share:**** Transaction fees on BRIJ-enabled purchases
- ****Data insights:**** Aggregated (anonymized) consumer behavior patterns

🏗️ HIGH-LEVEL ARCHITECTURE



📌 BRIJ CORE COMPONENTS

1. Protocol Adapter Manager

****Purpose:**** Translates BRIJ's unified API into platform-specific protocols

****Key Responsibilities:****

- Protocol version management (UCP v1, Copilot v2, ChatGPT v1)
- Request/response transformation
- Error handling and retry logic
- Rate limiting and throttling
- Capability negotiation

****Implementation:****

```typescript

```
interface ProtocolAdapter {
 platform: 'google_ucp' | 'microsoft_copilot' | 'openai_chatgpt';
 version: string;

 // Core methods
 discover(query: ProductQuery): Promise<Product[]>;
 createCart(items: CartItem[]): Promise<Cart>;
 initiateCheckout(cart: Cart): Promise<CheckoutSession>;
 processPayment(session: CheckoutSession): Promise<PaymentResult>;
 handlePostPurchase(order: Order): Promise<void>;

 // Capability checks
 supportsCapability(capability: string): boolean;
 getAvailablePaymentMethods(): PaymentMethod[];
 getMerchantOfRecordModel(): 'platform' | 'merchant';
}
```

```
class GoogleUCPAdapter implements ProtocolAdapter {
 async discover(query: ProductQuery): Promise<Product[]> {
 // Call UCP Discovery API
 const response = await ucpClient.products.search({
 query: query.text,
 filters: this.transformFilters(query.filters),
 user_profile: query.userContext
 });

 return this.transformUCPProducts(response.products);
 }
}
```

```

async initiateCheckout(cart: Cart): Promise<CheckoutSession> {
 // Register checkout capability with UCP
 const session = await ucpClient.checkout.create({
 cart_id: cart.id,
 merchant_id: cart.merchantId,
 capabilities: ['direct_offers', 'loyalty_rewards'],
 payment_handlers: ['google_pay', 'paypal']
 });

 return this.transformUCPSession(session);
}

```

```

class MicrosoftCopilotAdapter implements ProtocolAdapter {
 async discover(query: ProductQuery): Promise<Product[]> {
 // Analyze Copilot conversation for intent
 const intent = await copilotClient.conversation.analyzeIntent({
 conversation_id: query.conversationId,
 user_message: query.text
 });

 // Search via Microsoft Commerce API
 const products = await msCommerceClient.products.search({
 intent: intent,
 enterprise_id: query.enterpriseContext?.id
 });

 return this.transformMSProducts(products);
 }
}

```

```

class OpenAIChatGPTAdapter implements ProtocolAdapter {
 async discover(query: ProductQuery): Promise<Product[]> {
 // Use ChatGPT Plugins API
 const response = await openaiClient.plugins.shopify.search({
 query: query.text,
 filters: query.filters,
 user_preferences: query.userContext
 });

 return this.transformShopifyProducts(response.products);
 }
}
...

```

## ### 2. Merchant of Record Manager

**\*\*Purpose:\*\*** Handles merchant-of-record variations across platforms

**\*\*Platform Models:\*\***

- **\*\*Google UCP:\*\*** Retailer is merchant of record (MoR)
- **\*\*Microsoft Copilot:\*\*** Flexible (platform or retailer can be MoR)
- **\*\*OpenAI ChatGPT:\*\*** Platform is merchant of record

**\*\*Key Responsibilities:\*\***

- Tax calculation and remittance routing
- Regulatory compliance per jurisdiction
- Payment settlement timing
- Refund processing
- Dispute resolution

**\*\*Implementation:\*\***

```typescript

```
enum MerchantOfRecordModel {  
  PLATFORM = 'platform',  
  RETAILER = 'retailer',  
  HYBRID = 'hybrid'  
}
```

```
interface MerchantOfRecordConfig {  
  platform: Platform;  
  model: MerchantOfRecordModel;  
  taxResponsibility: 'platform' | 'merchant';  
  paymentSettlement: 'immediate' | 'delayed';  
  settlementDelay?: number; // days  
  disputeHandler: 'platform' | 'merchant';  
}
```

```
class MerchantOfRecordManager {  
  async determineModel(  
    order: Order,  
    platform: Platform  
  ): Promise<MerchantOfRecordConfig> {  
    const platformConfig = this.getPlatformConfig(platform);  
    const merchantPreferences = await this.getMerchantPreferences(order.merchantId);  
  
    // Google UCP: Always retailer  
    if (platform === 'google_ucp') {
```

```

return {
    platform: 'google_ucp',
    model: MerchantOfRecordModel.RETAILER,
    taxResponsibility: 'merchant',
    paymentSettlement: 'delayed',
    settlementDelay: 2,
    disputeHandler: 'merchant'
};
}

// Microsoft Copilot: Flexible based on merchant preference
if (platform === 'microsoft_copilot') {
    return merchantPreferences.preferPlatformMoR
        ? this.platformMoRConfig(platform)
        : this.merchantMoRConfig(platform);
}

// OpenAI ChatGPT: Always platform
if (platform === 'openai_chatgpt') {
    return {
        platform: 'openai_chatgpt',
        model: MerchantOfRecordModel.PLATFORM,
        taxResponsibility: 'platform',
        paymentSettlement: 'immediate',
        disputeHandler: 'platform'
    };
}

}

async calculateTaxes(
    order: Order,
    morConfig: MerchantOfRecordConfig
): Promise<TaxCalculation> {
    if (morConfig.taxResponsibility === 'platform') {
        // Platform handles tax calculation
        return this.delegateToPlatform(order, morConfig.platform);
    } else {
        // Use BRIJ tax engine
        return this.brijTaxCalculator.calculate(order);
    }
}
}
}
...

```

3. Payment Router & Processor Integration

****Purpose:**** Routes payments to appropriate processors based on platform and user preference

****Supported Payment Methods:****

- ****Google UCP:**** Google Pay (primary), PayPal, Card
- ****Microsoft Copilot:**** Microsoft Pay, Card, Enterprise Procurement
- ****OpenAI ChatGPT:**** Stripe (primary), Card, ACH

****Key Responsibilities:****

- Payment method selection and optimization
- PCI-DSS compliant tokenization
- Multi-currency support
- Payment retry logic
- Chargeback management

****Implementation:****

```typescript

```
interface PaymentRouter {
 selectOptimalPaymentMethod(
 user: User,
 platform: Platform,
 amount: Money
): Promise<PaymentMethod>;
```

```
 processPayment(
 payment: PaymentRequest
): Promise<PaymentResult>;
```

```
 handlePaymentFailure(
 payment: PaymentRequest,
 error: PaymentError
): Promise<PaymentResult>;
```

```
}
```

```
class BRIJPaymentRouter implements PaymentRouter {
 async selectOptimalPaymentMethod(
 user: User,
 platform: Platform,
 amount: Money
): Promise<PaymentMethod> {
 // Get user's saved payment methods
 const userMethods = await this.getUserPaymentMethods(user.id);
```

```

// Get platform-supported methods
const platformMethods = this.getPlatformPaymentMethods(platform);

// Filter to intersection
const availableMethods = userMethods.filter(m =>
 platformMethods.includes(m.type)
);

// Optimize selection based on:
// 1. User preference
// 2. Lowest fees
// 3. Fastest processing
// 4. Cashback/rewards
return this.optimizeSelection(availableMethods, amount);
}

async processPayment(payment: PaymentRequest): Promise<PaymentResult> {
 // Route to appropriate processor
 const processor = this.getProcessor(payment.method.type);

 try {
 const result = await processor.charge({
 amount: payment.amount,
 currency: payment.currency,
 source: payment.method.token,
 metadata: {
 order_id: payment.orderId,
 platform: payment.platform,
 brij_transaction_id: payment.brijId
 }
 });

 // Log successful payment
 await this.logPaymentSuccess(payment, result);

 return result;
 } catch (error) {
 return this.handlePaymentFailure(payment, error);
 }
}

async handlePaymentFailure(
 payment: PaymentRequest,
 error: PaymentError

```



```

): Promise<PaymentResult> {
 // Categorize error
 if (error.type === 'insufficient_funds') {
 // Suggest alternative payment method
 return this.suggestAlternative(payment);
 }

 if (error.type === 'card_declined') {
 // Retry with 3DS authentication
 return this.retryWith3DS(payment);
 }

 if (error.type === 'network_error') {
 // Retry with exponential backoff
 return this.retryWithBackoff(payment, 3);
 }

 // Unrecoverable error
 return {
 status: 'failed',
 error: error,
 suggestions: await this.generateFailureGuidance(error)
 };
}
}
}
...

```

---

## ## PLATFORM-SPECIFIC INTEGRATIONS

### ### Integration 1: Google Universal Commerce Protocol (UCP)

```

Protocol Version: v1.0
Documentation: https://developers.google.com/commerce/ucp
Launch Date: January 2026

```

#### #### A. Discovery & Product Search

```

Endpoint: `POST /v1/discovery`

BRIJ Implementation:
```typescript
class UCPSDiscoveryService {

```

```

async search(query: BRIJProductQuery): Promise<BRIJProduct[]> {
  const ucpRequest = {
    query: query.text,
    filters: {
      categories: query.categories,
      price_range: query.priceRange,
      availability: 'in_stock',
      merchant_ids: query.merchantIds
    },
    user_profile: {
      location: query.userLocation,
      preferences: query.userPreferences,
      purchase_history: query.purchaseHistory
    },
    capabilities: ['product_recommendations', 'similar_items']
  };

  const ucpResponse = await this.ucpClient.post('/v1/discovery', ucpRequest);

  // Transform UCP products to BRIJ unified schema
  return ucpResponse.products.map(p => this.transformProduct(p));
}

private transformProduct(ucpProduct: UCProduct): BRIJProduct {
  return {
    id: this.generateBRIJId('ucp', ucpProduct.id),
    platform: 'google_ucp',
    platformProductId: ucpProduct.id,
    title: ucpProduct.title,
    description: ucpProduct.description,
    price: {
      amount: ucpProduct.price.amount,
      currency: ucpProduct.price.currency,
      originalAmount: ucpProduct.original_price?.amount
    },
    merchant: {
      id: ucpProduct.merchant.id,
      name: ucpProduct.merchant.name,
      reputation: ucpProduct.merchant.rating
    },
    images: ucpProduct.images.map(img => img.url),
    availability: this.parseAvailability(ucpProduct.availability),
    capabilities: ucpProduct.supported_capabilities
  };
}

```

```
}  
}  
...
```

B. Checkout Flow

****UCP Checkout Sequence:****

1. Register checkout capability
2. User initiates purchase in AI Mode
3. UCP calls BRIJ callback URL
4. BRIJ creates cart and returns checkout URL
5. User completes checkout (on merchant site or in Google)
6. UCP confirms payment and notifies BRIJ

****BRIJ Implementation:****

```
```typescript
```

```
class UCPCheckoutService {
 async registerCheckout(merchant: Merchant): Promise<UCPRegistration> {
 return await this.ucpClient.post('/v1/capabilities/register', {
 merchant_id: merchant.ucpld,
 capabilities: [
 {
 type: 'checkout',
 version: '1.0',
 callback_url: `${BRIJ_API}/ucp/checkout/callback`,
 supported_features: [
 'direct_checkout',
 'cart_building',
 'dynamic_pricing',
 'loyalty_rewards'
]
 }
],
 payment_handlers: [
 {
 type: 'google_pay',
 merchant_account_id: merchant.googlePayAccountId
 },
 {
 type: 'paypal',
 merchant_account_id: merchant.paypalAccountId
 }
]
 });
 }
};
```

```

 }

 async handleCheckoutCallback(req: UCPCheckoutRequest): Promise<UCPCheckoutResponse> {
 // Validate UCP signature
 if (!this.validateUCPSignature(req)) {
 throw new Error('Invalid UCP signature');
 }

 // Create BRIJ cart
 const cart = await this.cartService.create({
 userId: req.user.id,
 items: req.selected_products.map(p => ({
 productId: p.product_id,
 quantity: p.quantity,
 platformPrice: p.price
 })),
 platform: 'google_ucp',
 sessionId: req.session_id
 });

 // Apply SCOOPS optimization
 const optimizedCart = await this.scoopsOptimizer.optimize(cart);

 // Return checkout URL
 return {
 checkout_url: `${BRIJ_CHECKOUT}/cart/${cart.id}`,
 estimated_total: optimizedCart.total,
 available_discounts: optimizedCart.appliedDiscounts,
 loyalty_rewards: optimizedCart.loyaltyRewards
 };
 }
}
...

```

#### #### C. Direct Offers Integration

**\*\*Feature:\*\*** Present exclusive discounts in AI Mode search results

**\*\*BRIJ Implementation:\*\***

```typescript

```

class UCPCDirectOffersService {
    async createOffer(merchant: Merchant, campaign: Campaign): Promise<UCPOffer> {
        // SCOOPS optimization to determine offer parameters
        const optimalOffer = await this.scoopsOptimizer.optimizeOffer({

```

```
merchant: merchant,
products: campaign.products,
targetAudience: campaign.audience,
budget: campaign.budget,
goals: campaign.goals
});
```

```
// Create UCP Direct Offer
```

```
return await this.ucpClient.post('/v1/offers/create', {
  merchant_id: merchant.ucpld,
  offer: {
    type: 'discount',
    value: optimalOffer.discountPercentage,
    minimum_purchase: optimalOffer.minimumPurchase,
    products: optimalOffer.applicableProducts,
    duration: {
      start: campaign.startDate,
      end: campaign.endDate
    },
    targeting: {
      user_segments: optimalOffer.targetSegments,
      keywords: optimalOffer.triggerKeywords,
      contexts: ['ai_mode_search', 'gemini_shopping']
    }
  },
  tracking: {
    callback_url: `${BRIJ_API}/ucp/offers/conversion`,
    utm_parameters: campaign.utmParameters
  }
});
```

```
}
}
...
```

Integration 2: Microsoft Copilot Checkout

Protocol Version: v2.0

Documentation: <https://docs.microsoft.com/copilot/commerce>

Launch Date: Q1 2025

A. Conversation-Driven Discovery

****Key Feature:**** Analyze Copilot conversation for shopping intent

****BRIJ Implementation:****

```typescript

```
class CopilotDiscoveryService {
 async analyzeConversation(conversationId: string): Promise<ShoppingIntent> {
 // Get conversation history
 const conversation = await this.copilotClient.conversations.get(conversationId);

 // Extract shopping signals
 const intent = await this.intentAnalyzer.analyze({
 messages: conversation.messages,
 user_context: conversation.user,
 enterprise_context: conversation.enterprise
 });

 return {
 intent_type: intent.type, // 'browse', 'compare', 'purchase'
 confidence: intent.confidence,
 extracted_requirements: intent.requirements,
 budget_range: intent.budget,
 timeline: intent.timeline,
 decision_factors: intent.factors
 };
 }
}

async searchBasedOnIntent(intent: ShoppingIntent): Promise<BRIJProduct[]> {
 // Build search query from conversational intent
 const query = this.intentToQuery(intent);

 // Search via Microsoft Commerce API
 const msProducts = await this.msCommerceClient.search({
 query: query,
 intent: intent,
 ranking_factors: intent.decision_factors
 });

 // Transform to BRIJ unified schema
 return msProducts.map(p => this.transformMSProduct(p));
}
}
```

#### B. Enterprise Integration

**\*\*Key Feature:\*\*** Connect to Azure AD and enterprise procurement systems

**\*\*BRIJ Implementation:\*\***

```typescript

```
class CopilotEnterpriseService {
  async authenticateUser(token: string): Promise<EnterpriseUser> {
    // Validate Azure AD token
    const azureUser = await this.azureAdClient.validateToken(token);

    // Get enterprise context
    const enterprise = await this.getEnterprisContext(azureUser.tenantId);

    return {
      userId: azureUser.id,
      email: azureUser.email,
      enterpriseld: enterprise.id,
      department: azureUser.department,
      budget_approval_limit: enterprise.getUserBudgetLimit(azureUser),
      procurement_rules: enterprise.getProcurementRules(azureUser.department)
    };
  }

  async checkProcurementApproval(
    user: EnterpriseUser,
    cart: Cart
  ): Promise<ApprovalStatus> {
    const rules = user.procurement_rules;

    // Check if purchase requires approval
    if (cart.total <= user.budget_approval_limit) {
      return { required: false, status: 'auto_approved' };
    }

    // Initiate approval workflow
    const approvers = await this.getApprovers(user, cart.total);
    const workflow = await this.procurementClient.createApprovalWorkflow({
      requester: user,
      cart: cart,
      approvers: approvers,
      reason: cart.businessJustification
    });

    return {
```

```

        required: true,
        status: 'pending',
        workflow_id: workflow.id,
        approvers: approvers,
        estimated_approval_time: workflow.estimatedTime
    };
}
}
...

```

C. Microsoft Pay Integration

****BRIJ Implementation:****

```

``typescript
class MicrosoftPayService {
    async processPayment(
        payment: PaymentRequest
    ): Promise<PaymentResult> {
        // Create Microsoft Pay payment request
        const msPayRequest = {
            amount: {
                value: payment.amount,
                currency: payment.currency
            },
            merchant: {
                id: payment.merchantId,
                name: payment.merchantName
            },
            user: {
                id: payment.userId,
                saved_payment_method_id: payment.method.microsoftPayId
            },
            metadata: {
                order_id: payment.orderId,
                brij_transaction_id: payment.brijId
            }
        };

        // Process via Microsoft Pay API
        const result = await this.msPayClient.payments.create(msPayRequest);

        // Handle enterprise billing
        if (payment.enterpriseContext) {
            await this.enterpriseBillingService.recordTransaction({

```



```
    enterprise_id: payment.enterpriseContext.id,  
    department: payment.user.department,  
    amount: payment.amount,  
    category: payment.category,  
    ms_pay_transaction_id: result.transaction_id  
  });  
}
```

```
  return result;  
}  
}  
...
```

Integration 3: OpenAI ChatGPT Instant Checkout

Protocol Version: v1.0

Documentation: <https://platform.openai.com/docs/plugins/commerce>

Launch Date: September 2025

A. Agentic Shopping Flow

Key Feature: Fully autonomous purchase decisions

BRIJ Implementation:

```typescript

```
class ChatGPTAgenticService {
 async handleShoppingRequest(
 conversationId: string,
 userMessage: string
): Promise<AgenticResponse> {
 // Analyze request using GPT-4
 const intent = await this.openaiClient.chat.completions.create({
 model: 'gpt-4',
 messages: [
 {
 role: 'system',
 content: 'You are a shopping assistant. Analyze the user message and extract purchase intent.'
 },
 {
 role: 'user',
 content: userMessage
 }
]
 });
 }
}
```

```

],
functions: [
 {
 name: 'extract_shopping_intent',
 parameters: {
 type: 'object',
 properties: {
 intent_type: { type: 'string', enum: ['browse', 'search', 'purchase'] },
 product_requirements: { type: 'object' },
 budget: { type: 'number' },
 urgency: { type: 'string', enum: ['immediate', 'soon', 'flexible'] }
 }
 }
 }
],
function_call: { name: 'extract_shopping_intent' }
});

```

```

const extracted_intent = JSON.parse(intent.choices[0].message.function_call.arguments);

```

```

// Search Shopify/Etsy via ChatGPT plugins

```

```

const products = await this.searchViaPlugins(extracted_intent);

```

```

// SCOOPS optimization

```

```

const optimized_products = await this.scoopsOptimizer.rank(products, extracted_intent);

```

```

return {
 recommended_products: optimized_products.slice(0, 5),
 reasoning: this.explainRecommendations(optimized_products),
 can_purchase_now: extracted_intent.intent_type === 'purchase'
};
}

```

```

async autonomousPurchase(

```

```

 user: User,

```

```

 product: Product,

```

```

 justification: string

```

```

): Promise<PurchaseResult> {

```

```

 // Safety checks

```

```

 const safetyCheck = await this.verifySafety({

```

```

 user: user,

```

```

 product: product,

```

```

 amount: product.price

```

```

 });

```

```

if (!safetyCheck.safe) {
 return {
 status: 'blocked',
 reason: safetyCheck.reason,
 user_confirmation_required: true
 };
}

// Process via Stripe (ChatGPT's payment processor)
const payment = await this.stripeService.charge({
 amount: product.price * 100, // cents
 currency: 'usd',
 customer: user.stripeCustomerId,
 description: `Autonomous purchase: ${product.title}`,
 metadata: {
 brij_transaction: true,
 gpt_conversation_id: justification,
 product_id: product.id
 }
});

return {
 status: 'completed',
 order_id: payment.id,
 receipt_url: payment.receipt_url
};
}
}
...

```

#### #### B. Shopify + Etsy Integration

**\*\*BRIJ Implementation:\*\***

```typescript

```

class ChatGPTMarketplaceService {
  async searchShopify(query: ProductQuery): Promise<BRIJProduct[]> {
    // Use ChatGPT's Shopify plugin
    const shopifyResults = await this.openaiClient.plugins.shopify.search({
      query: query.text,
      filters: {
        price_min: query.priceRange?.min,
        price_max: query.priceRange?.max,
        collection_id: query.categories?.[0]
      }
    });
  }
}

```

```

    },
    sort_by: 'relevance'
  });

  return shopifyResults.products.map(p => this.transformShopifyProduct(p));
}

```

```

async searchEtsy(query: ProductQuery): Promise<BRIJProduct[]> {
  // Use ChatGPT's Etsy plugin
  const etsyResults = await this.openaiClient.plugins.etsy.search({
    keywords: query.text,
    min_price: query.priceRange?.min,
    max_price: query.priceRange?.max,
    category: query.categories?.[0]
  });

  return etsyResults.listings.map(l => this.transformEtsyProduct(l));
}
}
...

```

C. Receipt & Expense Management

```

**BRIJ Implementation:**
```typescript
class ChatGPTReceiptService {
 async generateReceipt(order: Order): Promise<Receipt> {
 // Create detailed receipt
 const receipt = {
 order_id: order.id,
 date: order.completedAt,
 merchant: order.merchant.name,
 items: order.items.map(item => ({
 name: item.product.title,
 quantity: item.quantity,
 unit_price: item.price,
 total: item.quantity * item.price
 })),
 subtotal: order.subtotal,
 tax: order.tax,
 shipping: order.shipping,
 total: order.total,
 payment_method: order.paymentMethod.last4,
 pdf_url: await this.generatePDF(order)
 };
 }
}

```

```
};
```

```
// Auto-categorize for expense tracking
```

```
const category = await this.categorizeExpense(order);
```

```
// Store in user's ChatGPT context
```

```
await this.openaiClient.context.store({
```

```
 user_id: order.userId,
```

```
 type: 'receipt',
```

```
 data: { ...receipt, category: category }
```

```
});
```

```
return receipt;
```

```
}
```

```
async categorizeExpense(order: Order): Promise<string> {
```

```
 // Use GPT-4 for intelligent categorization
```

```
 const result = await this.openaiClient.chat.completions.create({
```

```
 model: 'gpt-4',
```

```
 messages: [
```

```
 {
```

```
 role: 'system',
```

```
 content: 'Categorize this purchase for expense tracking.'
```

```
 },
```

```
 {
```

```
 role: 'user',
```

```
 content: JSON.stringify({
```

```
 merchant: order.merchant.name,
```

```
 items: order.items.map(i => i.product.title),
```

```
 total: order.total
```

```
 })
```

```
 }
```

```
],
```

```
 functions: [
```

```
 {
```

```
 name: 'categorize_expense',
```

```
 parameters: {
```

```
 type: 'object',
```

```
 properties: {
```

```
 category: {
```

```
 type: 'string',
```

```
 enum: ['office_supplies', 'meals', 'travel', 'software', 'equipment', 'other']
```

```
 },
```

```
 subcategory: { type: 'string' },
```

```

 tax_deductible: { type: 'boolean' }
 }
 }
 },
],
function_call: { name: 'categorize_expense' }
});

return JSON.parse(result.choices[0].message.function_call.arguments).category;
}
}
...

```

---

## ## SECURITY & COMPLIANCE

### ### Authentication & Authorization

**\*\*OAuth 2.0 Flow:\*\***

```typescript

```

class BRIJAuthService {
  async authenticateUser(platform: Platform, authCode: string): Promise<BRIJUser> {
    // Exchange auth code for tokens
    const tokens = await this.exchangeAuthCode(platform, authCode);

    // Get user profile from platform
    const platformUser = await this.getPlatformUser(platform, tokens.access_token);

    // Create or update BRIJ user
    const brijUser = await this.userService.upsert({
      platform: platform,
      platform_user_id: platformUser.id,
      email: platformUser.email,
      name: platformUser.name,
      access_token: await this.encrypt(tokens.access_token),
      refresh_token: await this.encrypt(tokens.refresh_token),
      expires_at: tokens.expires_at
    });

    // Generate BRIJ session
    const session = await this.sessionService.create(brijUser.id);

    return { user: brijUser, session: session };
  }
}

```

```
}  
}  
...
```

Payment Security (PCI-DSS Level 1)

****Tokenization:****

```
```typescript  
class BRIJPaymentTokenizer {
 async tokenizePaymentMethod(
 method: RawPaymentMethod
): Promise<TokenizedPaymentMethod> {
 // Never store raw payment data
 // Always tokenize immediately
 const token = await this.stripeClient.tokens.create({
 card: {
 number: method.cardNumber,
 exp_month: method.expiryMonth,
 exp_year: method.expiryYear,
 cvc: method.cvc
 }
 });

 // Store only token, not raw card data
 return {
 token_id: token.id,
 last4: token.card.last4,
 brand: token.card.brand,
 exp_month: token.card.exp_month,
 exp_year: token.card.exp_year
 };
 }
}
...
```

### ### Data Encryption

**\*\*Implementation:\*\***

```
```typescript  
class BRIJEncryptionService {  
  // AES-256-GCM encryption  
  async encrypt(plaintext: string): Promise<string> {  
    const iv = crypto.randomBytes(16);  
    const cipher = crypto.createCipheriv('aes-256-gcm', this.masterKey, iv);  
    const encrypted = cipher.update(plaintext, 'utf8', 'hex');  
    const final = cipher.final('hex');  
    return encrypted + final;  
  }  
}
```

```

let encrypted = cipher.update(plaintext, 'utf8', 'base64');
encrypted += cipher.final('base64');

const authTag = cipher.getAuthTag();

// Return: iv + authTag + ciphertext (all base64)
return `${iv.toString('base64')}:${authTag.toString('base64')}:${encrypted}`;
}

async decrypt(ciphertext: string): Promise<string> {
  const [ivBase64, authTagBase64, encrypted] = ciphertext.split(':');

  const iv = Buffer.from(ivBase64, 'base64');
  const authTag = Buffer.from(authTagBase64, 'base64');
  const decipher = crypto.createDecipheriv('aes-256-gcm', this.masterKey, iv);
  decipher.setAuthTag(authTag);

  let decrypted = decipher.update(encrypted, 'base64', 'utf8');
  decrypted += decipher.final('utf8');

  return decrypted;
}
}
...

```

📊 MONITORING & OBSERVABILITY

Key Metrics

```

``typescript
interface BRIJMetrics {
  // Platform health
  platform_availability: {
    google_ucp: number;    // 0-100%
    microsoft_copilot: number;
    openai_chatgpt: number;
  };

  // Performance
  avg_response_time_ms: {
    discovery: number;

```



```

    checkout: number;
    payment: number;
};

// Business metrics
transaction_volume: number;
gmV: Money;
conversion_rate: number;

// Error rates
error_rate: {
    discovery_errors: number;
    checkout_errors: number;
    payment_errors: number;
};
}

```

```

class BRIJMonitoringService {
    async recordMetric(metric: string, value: number, tags: Record<string, string>) {
        await this.prometheusClient.gauge(metric).set(tags, value);

        // Alert if threshold exceeded
        if (this.exceedsThreshold(metric, value)) {
            await this.alerting.send({
                severity: 'critical',
                metric: metric,
                value: value,
                threshold: this.getThreshold(metric)
            });
        }
    }
}
...

```

🚀 DEPLOYMENT

Infrastructure

****Kubernetes Deployment:****

```

```yaml
apiVersion: apps/v1
kind: Deployment

```

```
metadata:
 name: brij-orchestrator
 namespace: scoops-production
spec:
 replicas: 10
 selector:
 matchLabels:
 app: brij-orchestrator
 template:
 metadata:
 labels:
 app: brij-orchestrator
 version: v1.0
 spec:
 containers:
 - name: brij
 image: scoops/brij:v1.0
 ports:
 - containerPort: 3000
 name: http
 env:
 - name: NODE_ENV
 value: production
 - name: GOOGLE_UCP_API_KEY
 valueFrom:
 secretKeyRef:
 name: brij-secrets
 key: google-ucp-api-key
 - name: MICROSOFT_CLIENT_ID
 valueFrom:
 secretKeyRef:
 name: brij-secrets
 key: ms-client-id
 - name: OPENAI_API_KEY
 valueFrom:
 secretKeyRef:
 name: brij-secrets
 key: openai-api-key
 resources:
 requests:
 cpu: 500m
 memory: 1Gi
 limits:
 cpu: 2000m
```

memory: 4Gi  
livenessProbe:  
 httpGet:  
 path: /health  
 port: 3000  
 initialDelaySeconds: 30  
 periodSeconds: 10  
readinessProbe:  
 httpGet:  
 path: /ready  
 port: 3000  
 initialDelaySeconds: 10  
 periodSeconds: 5  
...

---

## 📈 PERFORMANCE TARGETS

Metric	Target	Current	Baseline
API Response Time (p95)	<300ms	TBD	
Checkout Completion Rate	>94%	72%	(traditional)
Payment Success Rate	>98%	94%	(traditional)
Platform Availability	99.95%	TBD	
Cross-Platform Sync Latency	<500ms	TBD	
Error Rate	<0.5%	TBD	

---

\*\*End of BRIJ Technical Architecture Document\*\*

Version: 1.0  
Date: January 11, 2026  
Status:  Production Ready Design  
Next Review: March 2026